Communication IP Design for Software Register Programming and CDC Verification

A Thesis Submitted

as part of coursework requirements

for the Degree of

MASTER OF TECHNOLOGY

in

COMMUNICATION AND SIGNAL PROCESSING

by

Shinguingam Phaomei

(2025EEM1010)

under the guidance of

Dr. Sam Darshi



DEPARTMENT OF ELECTRICAL ENGINEERING

INDIAN INSTITUTE OF TECHNOLOGY ROPAR

September, 2026

Declaration

I hereby declare that the work presented in this mid-semester project report entitled **"Communication IP Design for Software Register Programming and CDC Verification"** is my original work carried out as part of the requirements for the degree of Master of Technology in Signal Processing and Communication at the Indian Institute of Technology Ropar.

The work presented in this report has not been submitted, either in part or in full, for the award of any other degree or diploma at any university or institution. All sources of information and references used in this work have been appropriately acknowledged.

Signature

Name:	Shinguingam Phaomei
Enrolment Number:	2025EEM1010
Program:	M.Tech
Department:	Electrical Engineering
Branch:	Communication and Signal Processing

Acknowledgement

I would like to express my sincere gratitude to Dr. Sam Darshi for his valuable guidance, support, and encouragement throughout this project work. I also thank the Department of Electrical Engineering, Indian Institute of Technology Ropar, for providing the necessary facilities and academic environment for carrying out this work. Finally, I am thankful to my family and friends for their continuous support and motivation.

Shinguingam Phaomei

2025EEM1010

Certificate

This is to certify that the project report titled **"Communication IP Design for Software Register Programming and CDC Verification"** submitted by **Shinguingam Phaomei (2025EEM1010)** is an original piece of work carried out under my supervision. As far as I am aware, no part of the work has been submitted to any other institution in support of a qualification elsewhere.

Signature of the Supervisor

Dr. Sam Darshi

Department of Electrical Engineering

Indian Institute of Technology Ropar, Rupnagar, Punjab 140001

Abstract

This report presents the design of a communication IP for software register programming using the AMBA APB protocol, along with a study of Clock Domain Crossing (CDC) verification. APB is a widely used low-power bus protocol in System-on-Chip designs, especially suitable for register-based communication due to its simple master-slave structure and low implementation complexity. In this work, an APB master-slave interface was developed in Verilog to support basic read and write transactions for software-controlled register access. The design was simulated and verified through waveform analysis to confirm correct bus behavior, transaction sequencing, and data transfer between master and slave modules. In addition to the APB implementation, CDC analysis was studied to understand the challenges associated with transferring signals between different clock domains. Unsafe CDC paths can lead to metastability, glitches, and functional failures that may not be detected through normal simulation alone. The CDC investigation helped identify common crossing-related issues and highlighted the need for synchronization techniques in digital hardware design. Rule-based CDC concepts such as glitch-prone crossings and unsynchronized scalar crossings were also studied using tool documentation.

Index Terms—APB, AMBA, Communication IP, Software Register Programming, CDC, Clock Domain Crossing, RTL Design, Verification, SoC.

Contents

Declaration i

Acknowledgement ii

Certificate iii

Abstract iv

1 Introduction 2

1.1 Introduction 2

1.2 Role of APB in Communication IP 3

1.3 Need for Software Register Programming 3

1.4 Importance of CDC Verification 3

1.5 Objectives of the Work 4

2 Literature Survey 5

2.1 APB as a Low-Power Communication Protocol 5

2.2 APB Design and Verification 5

2.3 Clock Domain Crossing (CDC) in Multi-Clock SoCs 5

2.4 Rule-Based CDC Verification 6

2.5 Summary of Literature 6

3 Problem Definition and Objectives 7

3.1 Problem Definition 7

3.2 Objectives 7

3.3 Scope of the Work 8

4 Methodology 9

4.1 Overview of the Approach 9

4.2 APB Protocol Methodology 9
4.3 APB Master-Slave Design Methodology 10
4.3.1 APB Master 10
4.3.2 APB Slave 10
4.4 RTL Implementation Methodology 11
4.5 Testbench Methodology 11
4.6 CDC Analysis Methodology 11

5 Implementation and Results **12**
5.1 APB Implementation 12
5.2 APB Transaction Flow 12
5.2.1 Write Operation 12
5.2.2 Read Operation 13
5.3 Simulation Results 13
5.4 CDC Analysis Results 14
5.4.1 `Ac_glitch03` 14
5.4.2 `Ac_unsync01` 14
5.5 Discussion of Results 14

6 Conclusion and Future Work **15**
6.1 Conclusion 15
6.2 Future Work 15
References 16

Chapter 1

Introduction

1.1 Introduction

Modern System-on-Chip designs integrate a large number of functional blocks such as processors, memories, peripherals, and control logic into a single device. To enable communication between these blocks, standardized bus protocols are used. Among these, the AMBA APB protocol is widely adopted for low-power, low-complexity communication with peripheral and register-based units. In many digital systems, software must configure hardware blocks by reading from and writing to memory-mapped registers. This process is commonly known as software register programming. For such applications, APB provides a suitable interface because of its simple handshake mechanism, low signal count, and easy implementation. Along with communication protocol design, modern digital systems must also address the issue of Clock Domain Crossing. In real SoCs, different modules may operate on different clocks, and signals crossing these domains must be carefully synchronized. If CDC is not handled properly, the design may suffer from metastability, glitches, or data corruption. This project focuses on two related but distinct topics:

- APB Slave/Master Interface for Communication IP for Software Register Programming
- CDC analysis in digital design

1.2 Role of APB in Communication IP

The AMBA APB protocol is a simple bus standard designed for peripheral communication. It is widely used for low-bandwidth applications where the priority is low power and low complexity rather than high-speed data transfer. APB is commonly used for accessing control registers in devices such as timers, UARTs, watchdogs, GPIO blocks, and other low-speed peripherals.

The main features of APB include:

- simple master-slave communication,
- two-phase transaction structure,
- no burst transfer support,
- low hardware overhead,
- suitability for register access.

1.3 Need for Software Register Programming

Software register programming is an essential part of embedded and SoC-based systems. Hardware modules usually expose configuration registers that control operation modes, status flags, interrupt enables, address selection, and data paths. These registers allow software running on a processor to interact with hardware in a controlled and standardized manner.

1.4 Importance of CDC Verification

CDC verification is used to identify potential problems when signals cross between different clock domains. It helps detect unsynchronized crossings, glitch-prone combinational paths, metastability risks, reconvergence issues, and signal transfer errors. CDC is important because a design may appear correct in simulation but still fail in real hardware due to timing-related issues.

1.5 Objectives of the Work

The main objectives of this project are:

- to understand the APB protocol and its role in communication IPs,
- to design an APB master and APB slave interface,
- to support software register programming through APB transactions,
- to simulate and verify APB read/write operations,
- to study CDC concepts and common CDC-related issues,
- to relate CDC analysis to reliable RTL design.

Chapter 2

Literature Survey

2.1 APB as a Low-Power Communication Protocol

The APB protocol is part of the AMBA family and is widely used in embedded systems for low-bandwidth peripheral access. It is designed to reduce complexity and power consumption while providing a reliable interface for register transfer. The paper on *APB-based memory communication* shows that APB can be used for efficient data exchange in low-power systems and highlights its simple structure, setup and access phases, and practical limitations in high-throughput scenarios.[1]

2.2 APB Design and Verification

The paper on AMBA AHB/APB interconnect verification emphasizes that bus-based designs require more than just simulation. Corner cases in transaction flow, handshaking, and configuration may remain hidden unless additional verification techniques are used. This supports the need for protocol-level validation in design projects involving APB interfaces.[2]

2.3 Clock Domain Crossing (CDC) in Multi-Clock SoCs

Modern SoCs integrate modules operating under multiple asynchronous clock domains, elevating Clock Domain Crossing (CDC) verification into a mandatory sign-off require-

ment [5]. Sundriyal and Gupta analyzed common CDC failures, including setup/hold violations that induce metastability, data loss during fast-to-slow crossings, and data incoherency on multi-bit buses [5]. Because traditional dynamic simulations fail to model non-deterministic metastability windows, static analysis and rule-based verification are essential [5, 6]. Li et al. developed automated static analysis techniques using Abstract Syntax Trees (AST) to extract clock trees, infer signal domains, and systematically verify crossing paths and synchronizers. [6]

2.4 Rule-Based CDC Verification

Static verification environments, such as Synopsys SpyGlass CDC, classify crossing integrity using formalized structural and functional rules.

- **Glitch Hazards (`Ac_glitch03`):** Detects combinational logic on synchronized crossings where differing signal arrival times, reconvergent paths, or multi-domain fan-ins generate transient pulses that can be captured by the destination clock. [5]
- **Unsynchronized Crossings (`Ac_unsync01`):** Identifies scalar signals entering a destination clock domain without a valid multi-flop synchronizer, synchronizing cell, or qualifier handshake, directly exposing receiving registers to metastability. [6]

2.5 Summary of Literature

From the literature, the following points are clear:

- APB is suitable for simple and low-power register communication.
- APB is commonly used in peripheral and control register access.
- APB designs require proper verification.
- CDC is a critical concern in multi-clock hardware systems.
- Both APB design and CDC analysis are essential for reliable SoC development.

Chapter 3

Problem Definition and Objectives

3.1 Problem Definition

Modern digital systems require a reliable method for software to communicate with hardware registers. APB is often used for this purpose because of its simplicity and low power consumption. However, implementing a correct APB interface requires proper handling of master-slave signaling, transaction states, and data transfer conditions. At the same time, many designs include multiple clock domains, creating potential CDC issues. Improper synchronization across clock boundaries can cause metastability and data corruption. Therefore, a communication IP design must be examined not only for functional correctness but also for timing and clock-domain safety.

3.2 Objectives

The main objectives of the project are:

1. To study the APB protocol and its operating principles.
2. To design APB master and slave modules in RTL.
3. To implement software register programming through APB transactions.
4. To verify correct APB read and write behavior using simulation.
5. To study the fundamentals of CDC and common crossing issues.
6. To understand why CDC verification is essential in modern digital designs.

3.3 Scope of the Work

The scope of the work includes:

- APB protocol study,
- RTL design of master-slave communication,
- simulation-based verification,
- basic CDC analysis and error understanding.

Chapter 4

Methodology

4.1 Overview of the Approach

The work was carried out in two main parts. The first part focused on the design and verification of an APB-based communication interface for software register programming. The second part focused on understanding CDC issues in digital designs and identifying common types of crossing-related errors.

4.2 APB Protocol Methodology

APB is a three-phase bus protocol consisting of:

- Idle phase
- Setup phase
- Access phase

The IDLE phase represents the default, inactive state of the bus when no data transfer is taking place. During the setup phase, the master selects the slave and provides address and control information. During the access phase, the actual read or write data transfer takes place. The slave responds using the ready and read-data signals.

The main APB signals used in the design are:

- `PCLK`: bus clock
- `PRESETn`: active-low reset
- `PSEL`: peripheral select
- `PENABLE`: enable signal
- `PADDR`: address bus
- `PWRITE`: indicates read or write
- `PWDATA`: write data bus
- `PRDATA`: read data bus
- `PREADY`: slave ready signal

4.3 APB Master-Slave Design Methodology

The APB communication IP was designed using a master-slave structure.

4.3.1 APB Master

The APB master is responsible for initiating transactions, generating address and control signals, selecting the slave, asserting write or read operation, and maintaining proper APB timing.

4.3.2 APB Slave

The APB slave is responsible for receiving address and control information, accepting write data, returning read data, and responding with ready status. The design logic follows the APB state sequence:

- Idle
- Setup
- Access

4.4 RTL Implementation Methodology

The RTL design was written in Verilog. The master and slave blocks were implemented as separate modules and connected through APB signals. The design supports basic register access using read and write operations.

4.5 Testbench Methodology

A testbench was created to verify the APB interface. The testbench generated clock and reset signals and applied read/write stimulus to the master interface. The simulation results were then observed using waveform analysis.

4.6 CDC Analysis Methodology

CDC analysis was studied separately to understand the risk of signal transfer across different clock domains. The main focus was on identifying unsafe crossings, understanding metastability, recognizing glitch-prone logic, and understanding unsynchronized paths.

The analysis showed that CDC issues often appear in:

- control signals,
- combinational fan-in paths,
- reconvergent logic,
- asynchronous crossings.

Chapter 5

Implementation and Results

5.1 APB Implementation

The APB interface was implemented as a simple communication block for software register access. The master generates APB transactions, while the slave responds to read and write requests.

The design supports:

- write transaction,
- read transaction,
- proper APB handshaking,
- transaction sequencing.

5.2 APB Transaction Flow

The APB transaction flow can be summarized as follows.

5.2.1 Write Operation

1. Master selects the slave using `PSEL`.
2. Master drives address on `PADDR`.
3. Master asserts `PWRITE = 1`.

4. Master enables the transfer using `PENABLE`.
5. Master drives write data on `PWDATA`.
6. Slave accepts the data and completes the transfer.

5.2.2 Read Operation

1. Master selects the slave.
2. Master drives the address.
3. Master sets `PWRITE = 0`.
4. Master asserts `PENABLE`.
5. Slave returns data on `PRDATA`.
6. Transaction completes when the slave is ready.

5.3 Simulation Results

The simulation waveform showed successful APB transactions. The following behavior was observed:

- correct state transition from Idle to Setup to Access,
- proper selection of slave,
- successful write transaction,
- successful read transaction,
- correct response from the slave.

5.4 CDC Analysis Results

CDC analysis highlighted common issues that may arise in multi-clock designs. The key error types studied were:

5.4.1 Ac_glitch03

This rule reports glitch-prone combinational logic in CDC paths. It may occur when crossing logic includes unstable fan-in or reconvergent paths.

5.4.2 Ac_unsync01

This rule reports unsynchronized scalar crossings for clock-domain signals. It indicates that a signal is crossing between domains without proper synchronization, which can cause metastability and functional failure.

5.5 Discussion of Results

The APB portion of the project confirmed that a simple master-slave bus can be used effectively for software register programming. The CDC study showed that separate clock-domain issues require special attention and should not be ignored during RTL development.

Chapter 6

Conclusion and Future Work

6.1 Conclusion

This project provided practical exposure to communication IP design using the AMBA APB protocol and to the fundamentals of CDC verification. The APB design demonstrated how a master-slave interface can be used for software register programming in a simple and effective manner. The simulation results confirmed correct read and write behavior. The CDC study helped develop understanding of signal transfer across clock domains and the importance of synchronization. It also showed how unsafe crossings can lead to metastability and functional errors. Overall, the work strengthened understanding of RTL design, bus communication, simulation-based verification, and CDC concepts, all of which are important in modern SoC development.

6.2 Future Work

Possible future extensions of this work include:

- supporting multiple APB slaves,
- integrating APB with a higher-level bridge,
- implementing CDC synchronizer circuits,
- extending verification with assertions,
- performing more detailed formal or semi-formal CDC checks.

References

[1] Prasad, Chaitanya Laxmi, and J. Charles Pravin. "Design and Implementation of APB Based Memory Communication." 2025 International Conference on Emerging Technologies in Engineering Applications (ICETEA). IEEE, 2025.

[2] Jaswal, Reshav, and Mamta Khosla. "Towards Comprehensive Semi-Formal Verification of AMBA AHB/APB Interconnects: Configuration Coverage, Mutation Testing, and Safety-Aware Design-for-Verification." 2026 International Conference on Intelligent Processing, Hardware, Electronics, and Radio Systems (CIPHER). IEEE, 2026.

[3] Sundriyal, Archit, and Vrinda Gupta. "Clock Reset Domain Crossing Issues and Verification Techniques for SoC Design." 2024 4th Asian Conference on Innovation in Technology (ASIANCON). IEEE, 2024.

[4] Li, You, et al. "A Method for Clock Domain Crossing Analysis of VHDL Code Based on Static Analysis." 2025 Asia-Europe Conference on Cybersecurity, Internet of Things and Soft Computing (CITSC). IEEE, 2025

[5] Synopsys, "Ac_glitch03: Reports clock domain crossings subject to glitches," *SpyGlass CDC Rule Reference Manual*.

[6] Synopsys, "Ac_unsync01: Reports unsynchronized clock domain crossing for scalar signals," *SpyGlass CDC Rule Reference Manual*.